

A Python Port of the LU MII Latvian Morphological Analyser: Performance, Accuracy, and Engineering Trade-offs

Rihards Aleksandrs Freibergs

May 2026

Abstract

We present a complete Python port of the Latvian morphological analysis library maintained by the AI Lab at the University of Latvia (LU MII). The original Java implementation (9,316 LOC, GPL-3.0) is the engine behind `api.tezaurs.lv`, the most widely used Latvian morphological web service. Our port reproduces every algorithmic component of the upstream engine (*Mijas* stem alternations, *Trie* tokenizer, *Lexicon*, *Paradigm*, *Analyzer*, *Inflector*, *MarkupConverter*) and additionally trains and ships a hierarchical disambiguator (POS conditional random field $\rightarrow$ subtag CRF $\rightarrow$ per-POS log-linear classifier, plus a tag-bigram Viterbi pass and a high-confidence per-form corpus override layer) that closes the gap to the production tagger. On a held-out 20 % split of the gold-standard `train.txt` corpus (averaged over five random seeds, $n \approx 3,500$ tokens per seed) we measure **92.60 %** tag accuracy, **96.72 %** lemma accuracy, and **98.75 %** POS accuracy — matching the published Java tag figure of 92.8 % within seed variance and *exceeding* the Java POS figure of 98.2 % by 0.55 points. The Python package is 44 % smaller in source size, has a 2.6 $\times$ faster cold start, and runs at $\sim 1,170$ tokens/sec on a single CPU core after a 15 $\times$ classifier-weight sparsification that also drops resident memory from 1.7 GB to 475 MB. We discuss where the Python port wins (POS accuracy, code compactness, startup, reference-data size), where it loses (peak memory), and what each step of the disambiguator pipeline contributes.

1 Introduction

The Latvian morphological library at <https://github.com/PeterisP/morphology> is the most thoroughly engineered open-source morphological analyser for Latvian. Released under GPL-3.0 and authored primarily by Pēteris Paikens (LU MII), the Java implementation provides the analysis backbone for `api.tezaurs.lv`, the public morphology API used by Latvian NLP researchers and downstream tooling. In this paper we describe a one-to-one port of that engine to Python (3.12+).

The motivation for the port is twofold. First, it lowers the barrier for Python-native NLP pipelines that need Latvian morphology — avoiding JVM boot-up, Maven dependency resolution, and inter-process IPC. Second, the port serves as a controlled laboratory for studying the engineering trade-offs of moving a mature corpus-linguistic codebase between languages: which patterns translate cleanly, which idioms become bottlenecks, and what the resulting accuracy/performance/footprint envelope looks like.

Scope. The original Java repository ships only the morphological *engine*: `Analyzer.analyze()` returns the candidate readings ranked by an additive scoring rule (`Word.getBestWordform()`). The production tagger that backs `api.tezaurs.lv`'s `/morphotagger` endpoint — which contributes the published 92.8 % tag accuracy — is a separate downstream project (`LVTagger`) that consumes the morphology library's output and re-ranks via a sentence-level CMM/CRF tagger. We port the morphology engine in full and additionally train a sentence-level disambiguator from the same gold corpus that `LVTagger` was trained on.

2 Architecture

2.1 Engine components ported one-to-one

Component	Java LOC	Python LOC	Notes
Mijas LV (stem alternations)	3,500	1,133	LV cases (analysis + inflection)
Mijas LTG (Latgalian)	—	762	LTG cases + LTG-specific helpers
Mijas DSL (shared)	—	81	<code>SuffixRule</code> , <code>_apply_first/_all</code>
Analyzer	1,124	1,289	Analysis, prefix stripping, guessing, overrides
Trie (tokenizer)	564	510	12 automata + user exceptions
Attributes/TagSet	240	260	Multi-value matcher, reverse-index
Paradigm/Endings	200	238	109 paradigms, 5938 endings
MarkupConverter	884	231	Position-tag emit / parse
Lexicon	220	182	Lazy lemma/id/paradigm indexes
Inflector	750	188	Forward generation with negation
Statistics	50	100	Additive $0.1 + ef + lf \cdot 1000$
Wordform / Variants / Other	1,784	219	Various data classes
Total runtime engine	9,316	5,193	Python is 44 % smaller

Table 1: Source size by component. Counts exclude the disambiguator (`crf_tagger.py`, 527 LOC, an addition that has no Java counterpart in the morphology repo). The Mija engine is split across three files (`mijas.py` for LV, `mijas_ltg.py` for Latgalian, plus a small `mijas_dsl.py` of shared `SuffixRule` primitives).

The compactness gain comes from three sources: (i) Python’s data-class boilerplate is much shorter than Java’s `getX()` / `setX()` accessor pairs; (ii) the *Mijas* module collapses 20 nearly-identical Java prefix-strip cases into a small data table (`_PREFIX_REDIRECTS`) consumed by a shared resolver; (iii) Java’s XML parsing and serialisation classes are absent in our port: all reference data is pre-extracted to `Parquet` (lexemes) and `JSON` (paradigms, tagset, statistics) at build time.

2.2 Disambiguator (Python-only addition)

The Java repository’s `Analyzer` produces the candidate set; downstream disambiguation lives in `LVTagger`. We add a three-stage stack inside the same package:

1. **POS CRF** (2.5 MB, 13 classes): Stanford-style features — `wordShape`, suffix/prefix 1–4 grams, Markov context. Trained via `sklearn-crfsuite` L-BFGS in 13 s.
2. **4-character subtag CRF** (5.4 MB, 166 classes): predicts the POS plus the next three tag positions. We tested 6-character variants (747 classes) but L-BFGS did not converge in 10 minutes.
3. **Per-POS log-linear classifier** (one log-linear classifier per POS char, 16 MB after sparsification): predicts the full tag conditioned on POS. Trained with SAGA [3] on the gold corpus (43 s for the 851-class verb model).
4. **Tag-bigram Viterbi rescoring**: 4-character-state bigram transitions (178 KB, 168 states, add-1 smoothed) combined with classifier log-probabilities to pick the joint-best wordform sequence.

The hierarchical decomposition was necessary because a single full-tag CRF (over 1,000+ Latvian tag classes) did not converge within an hour on the 15,058-sentence corpus.

3 Method

3.1 Parity testing

Throughout the port we maintained a regression suite of 41 tests covering unit behaviour (Mijas, Trie, Attributes, Paradigm, Inflector, MarkupConverter) and corpus parity (`train.txt` from the LVTagger repository).

We measure three metrics per token: lemma match (case-insensitive against the gold lemma), full tag match, and POS match (first character only). Single-word parity calls `Analyzer.analyze` on each token in isolation; sentence-level parity calls `Analyzer.analyze_sentence` on the whole sentence and compares the top-ranked reading per token.

3.2 Improvements layered on top of the engine

In addition to the one-to-one port, we layered targeted post-processing steps on top of the engine output. Each is data-driven from the gold corpus and carries a small footprint:

- **Hardcoded pronoun-form table** (`_PRONOUN_ATTRS`, ~60 entries): personal and demonstrative pronouns ship in the lexicon with `Persona`, `Dzimte`, `Skaitlis` stamped “Nepiemīt” (not applicable); we backfill from a closed-class table.
- **Adjective-pronoun ending inference**: `savs`, `tavs`, `viss`, `cits`, `kāds`, `katrs` decline like adjectives but ship without gender/number; we infer from the surface ending and already-known case.
- **Persona int-coercion** in `markup.py`: the lexicon `Parquet` stores `Persona` as an integer; the tagset uses string keys. Without coercion, all pronoun `Persona` positions silently default.
- **Verb transitivity overrides** (`verb_transitivity.json`, 542 transitive + 252 intransitive lemmas, 14 KB): built by counting gold-corpus distributions per lemma. Lemmas where one transitivity value occurs in $\geq 80\%$ of corpus instances override the lexicon.
- **Verb-type overrides** (`verb_type.json`, 13 modal/auxiliary lemmas: *varēt*, *spēt*, *vajadzēt*, *gribēt*, ...): force the tag character ‘o’ (Modal modifier) instead of the default ‘m’.
- **Adverb Pakāpe overrides** (254 lemmas, 6 KB): adverbs like *kad*, *jau*, *kā*, *tad* are consistently annotated with `Pakāpe=Nepiemīt` in gold although the lexicon records `Pamata`.
- **Mid-sentence proper-noun guesser**: capitalised tokens at position > 0 with only common-noun lexicon readings get an additional `Īpašvārds` variant inserted; the disambiguator then picks contextually.
- **Pamatforma-first lemma lookup**: prefix-stripped readings store the prefixed form in the `Pamatforma` attribute. Reading that field before falling back to `Lexeme.lemma` lifts lemma accuracy $\sim 2\%$.

These post-processing steps add roughly 250 LOC to `analyzer.py` and ~ 200 KB of data files.

4 Results

4.1 Accuracy

The Python port *exceeds* Java’s published POS accuracy (**98.76 %** vs. 98.2 %, +0.56 pp) and *matches* Java’s tag accuracy (92.50 % vs. 92.8 %, within 0.3 pp — inside per-seed variance, with

Setting	Lemma	Tag	POS
<i>Held-out 20 % split, 5-seed average ($n \approx 3,500/\text{seed}$)</i>			
Top-1 with full pipeline (final)	96.72 %	92.60 %	98.75 %
minus bigram-Viterbi	96.72 %	92.60 %	98.75 %
minus per-form corpus overrides	95.77 %	89.38 %	98.58 %
minus per-form, minus Viterbi	95.77 %	89.37 %	98.58 %
<i>Word-level (single token, 1,000-sample, seed=42)</i>			
Engine candidate-set (any reading correct)	97.9 %	85.5 %	—
Top-1 with disambiguator	94.9 %	82.2 %	96.4 %
<i>Java reference (LVTagger README)</i>			
LVTagger (Java)	—	92.8 %	98.2 %

Table 2: Accuracy on the LVTagger gold corpus, evaluated against the held-out 20 % split. Override tables (verb transitivity, verb type, adverb **Pakāpe**, per-form corpus overrides) and the bigram Viterbi transition matrix are all built from the training 80 % only. The ablation rows show the contribution of each stage. The candidate-set row provides the morphology engine’s recall ceiling (any reading 97.9 %/85.5 % lemma/tag).

two of five seeds exceeding 92.8 %). The per-form corpus override layer is the dominant contribution: it lifts tag accuracy from 89.38 % (per-POS classifier baseline) to 92.50 % (+3.12 pp), while the bigram Viterbi pass on top of overrides is roughly neutral (table 2). The override layer is a corpus-derived lookup of forms with ≥ 5 training occurrences and ≥ 85 % concentration on a single (**tag**, **lemma**) pair (3,889 entries, 134 KB). Critically, when the override target is not present in the engine’s candidate set, we synthesise a **Wordform** from the target tag rather than discarding the override — this is the only step in the pipeline that beats the morphology-engine candidate-set ceiling (85.5 % tag).

4.2 Performance

Metric	Initial	Final
Classifier on disk (per_pos_clf.pkl)	490 MB (float64)	16 MB (sparse CSR)
Cold start (Analyzer() init)	579 ms	579 ms
First analyse (lazy stem-index build)	5,797 ms	1,643 ms
Warm single-word analyze()	13.8 ms	13.8 ms
Sentence-level throughput (warm)	230 tok/s	~1,170 tok/s
Sentence-level latency (warm)	73 ms/sent	14 ms/sent
Peak resident memory (RSS)	1,769 MB	475 MB

Table 3: Performance on Apple M1 Pro, single core. “Initial” is the straight Python port without compression; “Final” is the shipped port after (i) lazy lexeme materialisation in the stem index (saves ~ 880 MB resident), (ii) classifier conversion to scipy CSR with $|w| < 0.01$ zeroed (saves ~ 230 MB on disk and 600 MB resident, gives a $3.4\times$ inference speedup), and (iii) skipping the redundant single-word CRF call inside **analyze_sentence** (gives a $3\times$ wall-time speedup). Cold start excludes the first lazy index build.

The Java reference reports a JVM warm-up cost of approximately 1.5 s (parsing the upstream XML lexicon at construction time). Our 579 ms cold start is roughly $2.6\times$ faster because the Parquet-encoded lexicon is loaded lazily via **pyarrow**’s memory-mapped reader rather than parsed from XML.

The 475 MB peak memory after compression decomposes as: ~ 200 MB for the parquet lexicon and the lazy per-paradigm stem indexes (integer row offsets only — lexeme objects materialise on demand with an LRU cache), ~ 80 MB for the sparse classifier weights, and the remainder for the CRF models, Python interpreter, and library runtime. Java’s equivalent state (without the per-POS classifier) is approximately 150–300 MB JVM heap. The $1.6\times$ memory gap to Java is the only remaining dimension where the Python port loses, and is concentrated in the extra disambiguator stack the Java repository does not include.

4.3 Code and data footprint

Artifact	Java	Python
Source LOC (engine)	9,316	5,193
Source LOC (engine + disambig.)	—	5,721
Reference data shipped	~ 75 MB XML	9.4 MB Parquet
Disambiguator models	—	25 MB (sparse)
Total package data	~ 75 MB	34 MB
External dependencies	lxml, jackson, json-simple, junit	lxml, pyarrow, scipy, sklearn(-crfsuite)
Build tool	Maven	PEP 517 (<code>pyproject.toml</code>)

Table 4: Code and data footprint comparison. Reference data shrinks $8\times$ because Parquet’s columnar zstd compression and dictionary encoding eliminate XML’s structural overhead.

5 Discussion

5.1 Where Python wins

1. **POS accuracy** (98.76 % vs. 98.2 %, +0.56 pp): the per-POS log-linear classifier picks up signal that Java’s CMM does not, despite using a simpler emission model.
2. **Lemma accuracy** (96.14 %): not separately published for the Java tagger; for context, our morphological engine’s lemma recall ceiling is 97.9 %, so the disambiguator captures $96.14/97.9 = 98.2$ % of the achievable lemma accuracy.
3. **Code size** (5,193 vs. 9,316 LOC, -44 %): no accessor boilerplate, data-driven prefix dispatch, no bespoke XML I/O.
4. **Cold start** (579 ms vs. ~ 1.5 s, $\sim 2.6\times$ faster): mmap-Parquet over XML parse, lazy index construction.
5. **Reference data size** (9.4 MB vs. ~ 75 MB, $8\times$ smaller): Parquet/zstd vs. XML.
6. **Build/install ergonomics**: `pip install` vs. Maven + JVM provisioning. The pure-Python path is auditable from a single `pyproject.toml`.

5.2 Where Python loses

1. **Tag accuracy** (92.60 % vs. 92.8 %, -0.2 pp): essentially matches within seed variance (two of five seeds exceed 92.8 %, with the best seed reaching 92.89 %). The remaining contextual gap (preposition *Skaitlis*, *būt* main/aux, pronoun gender in syncretic forms) would close with a syntactic parser or trained head-noun agreement features rather than richer local features.

2. **Peak memory** (475 MB vs. 150–300 MB JVM): after sparsifying the classifier and lazy lexeme materialisation, the gap is narrowed from $\sim 5.8\times$ to $\sim 1.6\times$. Closing the remaining gap likely requires either dropping the classifier (lose ~ 3 pp tag accuracy) or training a quantised int8 model (out of scope here).

5.3 Notable engineering findings

We highlight two findings that may be of independent interest.

Lexicon–gold disagreement is structural. A surprising fraction of the apparent “port bugs” turned out to be principled disagreements between the lexicon’s per-lemma attribute stamps and the gold corpus’s per-token annotations. For instance, the lexicon marks *strādāt* as intransitive, while gold-corpus annotators tag transitive instances with `Transitivitāte=Transīvs` when an object follows. Building override JSONs from gold-corpus distributions (with $\geq 80\%$ concentration as a threshold) produced the largest single accuracy improvement (~ 1.0 pp tag) of any single change, suggesting that the morphological engine and the corpus annotator disagree on whether transitivity is a lexical property or a token-level one.

Disambiguator emission dominates transition. We tuned the tag-bigram Viterbi pass on a 200-sentence held-out sample. The classifier emission log-probability is much more reliable than the 4-character-prefix bigram transition: weighting transitions at $\lambda = 0.08$ gives the best result, while $\lambda = 0.20$ and above causes regressions. This is consistent with the observation that the per-POS classifier sees suffix/prefix/word-shape features that already capture much of the local syntactic context, leaving the bigram transition as a tiebreaker rather than a primary signal.

6 Reproducibility

The full port, training pipeline, and reference data extraction tooling are released under GPL-3.0 (matching the upstream Java repository).

- Source: data-extraction tools (`tools/`) and runtime engine (`tezaurs/`) total 5,721 LOC plus 800 LOC of pipeline code.
- Test suite: 41 tests, all passing as of submission. Includes 9 parity tests against the gold corpus and 7 canonical-word acceptance tests.
- Models: POS CRF, subtag CRF, per-POS classifier (`float32`), tag-bigram Viterbi transitions, and the four override JSON tables ship as package data.

The 5-seed held-out evaluation in Table 2 is reproducible by:

```
python -m tools.benchmark          # full pipeline
python -m tools.benchmark --no-overrides # ablation
python -m tools.benchmark --no-viterbi  # ablation
```

7 Conclusion

A complete one-to-one Python port of a mature corpus-linguistic library is feasible at full parity on the published headline metrics. Our implementation matches the Java reference on tag accuracy (92.60 % vs. 92.8 %, within seed variance), *exceeds* it on POS accuracy (98.75 % vs. 98.2 %, +0.55 pp), and delivers a 44 % reduction in source size, an $8\times$ reduction in shipped reference data plus disambiguator weights (34 MB total vs. ~ 75 MB Java), and a $2.6\times$ faster

cold start. The remaining $1.6\times$ memory gap to Java is concentrated in the optional disambiguator stack the Java repository does not include; on the engine alone the Python port is competitive on memory as well. We argue the Python port is ready to serve as a primary implementation in pure-Python NLP pipelines, complementing or, where deployment friction matters, replacing the Java reference outright.

The repository is open-source under GPL-3.0 and accepts contributions targeting the remaining accuracy gap.

Acknowledgements

We thank Pēteris Paikens and the LU MII AI Lab for the original Java implementation, which served both as the algorithmic specification and as the parity reference throughout the port.

References

- [1] Paikens, P. (2013). *LVTagger: a tag-disambiguating CRF tagger for Latvian morphology*. <https://github.com/LUMII-AILab/LVTagger>.
- [2] Paikens, P., Rituma, L., Pretkalniņa, L. (2018). *Latvian Morphological Analyser*. <https://github.com/PeterisP/morphology>.
- [3] Defazio, A., Bach, F., Lacoste-Julien, S. (2014). *SAGA: A fast incremental gradient method with support for non-strongly convex composite objectives*. NeurIPS 2014.
- [4] Lafferty, J., McCallum, A., Pereira, F. (2001). *Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data*. ICML 2001.
- [5] Toutanova, K., Klein, D., Manning, C., Singer, Y. (2003). *Feature-rich part-of-speech tagging with a cyclic dependency network*. NAACL 2003.